🧠 **Overall System (Advanced but Simple Soch)**

1. User CV deta hai
2. Tum usay **clean + safe (PII hide)** karte ho
3. Usko **vector memory** mein daalte ho
4. AI us CV ko samajh kar **questions banata hai**
5. Interview hota hai
6. AI feedback deta hai

🏗️ **FINAL TECH STACK (Tumhara Setup)**

- Backend: **Python + FastAPI**
- AI Framework: **LangChain**
- LLM: **OpenAI GPT-4.1 Mini**
- Vector DB: **ChromaDB** (start)
  👉 Later: Pinecone / Weaviate
- Embeddings: OpenAI embeddings
- File parsing: PyPDF / Docx
- Security: PII detection + masking

🪜 **COMPLETE FLOW (A → Z)**

🟢 **STEP 1: Environment Setup**

Output:

Project ready

pip install fastapi uvicorn langchain openai chromadb pydantic python-multipart

---

🟢 **STEP 2: FastAPI Server**

Output:

API chal rahi ho

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Server running"}
```

Run:

```
uvicorn main:app --reload
```

🟢 **STEP 3: CV Upload API**

Output:

File receive ho

```
from fastapi import UploadFile, File

@app.post("/upload")
async def upload_cv(file: UploadFile = File(...)):
    contents = await file.read()
    return {"filename": file.filename}
```

🟢 **STEP 4: CV TEXT Extract**

Output:

Raw text mil jaye

👉 PDF:

```
pip install pypdf
from pypdf import PdfReader

def read_pdf(file_path):
```

```
    reader = PdfReader(file_path)
    text = ""
    for page in reader.pages:
        text += page.extract_text()
    return text
```

🟢 **STEP 5: 🔐 PII DETECTION + MASKING (VERY IMPORTANT)**

Output:

Sensitive data hide ho jaye

🧠 **PII kya hota hai?**

- Email
- Phone number
- CNIC
- Address

Simple Masking:

```
import re

def mask_pii(text):
    text = re.sub(r'\S+@\S+', '[EMAIL]', text)
    text = re.sub(r'\b\d{11}\b', '[PHONE]', text)
    return text
```

👉 Advanced:
Use NER models (spaCy / Presidio)

🟢 **STEP 6: Text Chunking**

Output:

Text chhote pieces mein divide ho

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_text(masked_text)
```

🟢 **STEP 7: Embeddings + Vector DB**

Output:

Searchable memory ban jaye

```
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma

embeddings = OpenAIEmbeddings()

db = Chroma.from_texts(chunks, embeddings)
```

🟢 **STEP 8: ADVANCED RAG (🔥 CORE INTELLIGENCE)**

Output:

Relevant CV parts AI ko milen

```
retriever = db.as_retriever(search_kwargs={"k": 3})
```

```
docs = retriever.get_relevant_documents("skills and experience")
```

👉 Samajh:
AI poora CV nahi dekhta
Sirf **relevant parts** dekhta hai → smart ban jata hai

🟢 **STEP 9: LLM Call (GPT-4.1 Mini)**

Output:

Questions generate ho

```
from langchain.chat_models import ChatOpenAI

llm = ChatOpenAI(model="gpt-4.1-mini")

response = llm.invoke(f"""
Based on this CV:
{docs}

Generate 5 technical interview questions
""")
```

🟢 **STEP 10: Interview Chain (LangChain Magic)**

Output:

Conversation flow ban jaye

```
from langchain.chains import RetrievalQA

qa_chain = RetrievalQA.from_chain_type(
```

```
    llm=llm,
    retriever=retriever
)
```

🟢 **STEP 11: Answer Evaluation**

Output:

Score + feedback

```
evaluation = llm.invoke(f"""
Question: {question}
Answer: {user_answer}

Evaluate:
- Score (1-10)
- Strengths
- Weaknesses
""")
```

🟢 **STEP 12: Storage (Masked Data Only)**

Output:

Safe database storage

👉 Store only:

- masked CV
- embeddings
- answers

👉 NEVER store:

- raw PII

🔐 DATA COMPLIANCE (🔥 MUST)

Follow karo:

✅ Mask before storing
✅ Encrypt DB (later)
✅ Logs mein PII avoid karo
✅ Access control add karo

🚀 ADVANCED RAG (LEVEL UP)

Add karo:

1. Metadata filtering

{"skill": "python"}

2. Hybrid search

- Vector + keyword

3. Re-ranking

- Best chunks select karo

🧠 FINAL FLOW (SUPER SIMPLE)

Upload CV

↓

Extract text

↓

Mask PII 🔐

↓

Chunking

↓

Embeddings

↓

Vector DB

↓

Retriever

↓

LLM (Questions)

↓

User Answers

↓

LLM (Evaluation)

🎯 **FINAL ONE-LINE**

👉 "Clean CV → Safe bana → Vector memory → AI se smart interview"